

Behind the Code: Uncovering Trends in Open Source Development

Introduction

Open source software (OSS) development has been a phenomenon of great interest for both individuals and organizations. Prior academic research has identified factors including ideology, altruism, reputation enhancement, and reciprocity as primary motivators for individual contributors (¹Von Krogh et al 2012). Organizations, conversely, tend to be driven by talent recruitment, accelerated innovation cycles, and community signaling (² Davenport and Bean 2022).

GitHub is the biggest platform that hosts Open-source software development projects. The aim of this project is to conduct an empirical analysis of real-world OSS contribution patterns leveraging data sourced from the GitHub platform. This study represents the practical application of data mining techniques learned by the authors in the Data Mining course.

Specifically, our analyses include the examination of geographical contribution trends within the United States, hypothesis testing on the relationships between programming languages and user contribution levels, and the assessment of temporal update patterns.

Through rigorous descriptive, visual, and statistical analyses, we document in this report meaningful insights into open-source developer motivations and behaviors. We hope that readers will find these GitHub contribution analyses as revelatory as we did in undertaking this project.

Research questions: Objective and Rationale

This project aims to address the following key research questions using the data of user contribution on the GitHub platform:

What are the geographical trends in OSS contributions across different US states?

Rationale: One of our main curiosity was to check whether regional factors such as the presence of Universities, innovation ecosystems, and industry clusters affect the OSS contribution. Understanding the contribution patterns across states allowed us to identify innovation hubs and correlate them with demographic, economic, and technology-industry factors.

How does OSS contribution level vary across programming languages?

Rationale: Intuitively, we expect certain languages used in OSS projects to be more popular than others. We also expect this popularity to change over time. The variation in contributions can be due to requirement and complexity of applications, learning curves, or intrinsic interests of developers. Analyzing differences helped to uncover these patterns.

How do OSS contributions vary among organizational and individual contributors?

Rationale: Developers' contribution to OSS could be acting as individual contributors or be part of an organization. While individual contributions indicate higher personal motivation and feeling of reciprocity to the Open-source movement, organizational contributions are expected to be driven more by job responsibility. Also, we would expect organizational repositories to exhibit higher productivity and promptness (responding to open issues) because of resource availability and collaboration. Testing hypotheses related to this variation reveals dynamics of these groups.

Data

Data Description

The data utilized in this analysis was sourced from GitHub's public API, accessed programmatically using Python. It includes granular information on over 500,000 open-source software projects spanning the period of 2014-2022.

Specific variables collected for the purpose of this analysis include:

- Project attributes: name, creation date, owner type, primary coding language
- Activity metrics: commits, issues, watchers, and repository status(active/archived)
- User information: location, type (individual vs organization)
- Temporal information: date of last push, update times

To ensure a well-balanced representation of user and organizational repositories in our analysis, we initially had a dataset containing contributions from over 4,435,290 unique repository details. However, in order to fix the imbalance, we used stratified sampling techniques. As a result, the dataset was reduced to 34,100 records. This reduction was necessary to ensure that both users and organizations were properly analyzed during Exploratory Data Analysis.

Statistical methods and software

With the goal of uncovering insightful trends and testing hypotheses related to open source development, the following statistical techniques and tests were leveraged:

- Descriptive statistics (means, distributions)
 - Descriptive statistics involve calculating summary measures such as mean, median, and standard deviation to provide the summary statistics of the dataset.
- Data visualization (plots, heat maps)
 - Data visualization techniques, including different types of plots and heat maps were used to represent the overview of the dataset.
 - Plots such as bar charts, scatterplots, violin plots, and line graphs were used to explain the trends and patterns in the dataset.
- Cluster analysis (K-means, DBSCAN, Hierarchical clustering)
 - The Standard Scaling library was used to normalize the dataset.
 - Various clustering algorithms were used to segment the data into users and organizations, to ensure that the dataset is properly balanced.
 - Silhouette score was used to assess the accuracy of the clustering algorithms.
- Hypothesis testing (ANOVA) (⁶Chandrankantha 2014)
 - Analysis of variance is used to analyze the group mean differences in the dataset.
 - Hypothesis testing was applied to determine whether the users and organizational groups exhibited significant differences or similarities.
- Temporal pattern analysis
 - Temporal analysis was used to understand the trends and patterns in the OSS projects activities. Techniques such as time series analysis were used to uncover the insights related to OSS projects.

Tools and Libraries: Python was the primary programming language used for the entire data analysis. Key libraries such as Pandas, matplotlib, seaborn, and scipy were used for data manipulation, visualization, and statistical analysis.

Methods

Data Collection

1. Selection Criteria:

User Diversity: our goal was to extract diverse representation of users and organizations across different states in the USA.

Random Sampling: Random Sampling was used to extract 1000 user names from each state in the USA to ensure fairness and to minimize the selection bias. (⁵Yu 2010)

2. API Rate Usage and Issues:

Rate Limits: GitHub allows 5000 requests per hour for authenticated users and 60 requests for non-authenticated requests.

Authentication: GitHub personal access tokens were used for creating an authenticated connection between the python script file and GitHub API.

3. **API Data Handling:**

We obtained data from the GitHub API in JSON format and filtered the necessary features including branch names, user type, languages used, contributions, repository name, description, creation and updation timestamps, and status. This filtered data was then converted into a Pandas DataFrame. Once the entire user repository dataset was collected, it was exported to a CSV file for further analysis.

Initial requests returned summary statistics, which we used to target information-rich repositories with substantial contributor activity. Further targeted queries extracted granular commit logs and temporal data meeting thresholds for our analysis suitability.

In total, we scanned over 443529 projects, with a final sample of approximately 34350 repositories meeting the inclusion criteria. This extraction process was optimized through iterative querying to balance data quality and volume considerations.

We followed our class notes, the course textbook (³ Han et al 2006), and various academic sources as our conceptual references throughout our analyses.

Data Preparation

Before analysis, we undertook extensive data preparation activities:

- Handling of missing values using deletion or imputation (mean/median/mode)
 - Missing values in the dataset were handled based on the data type of column they belonged to.
 - Columns related to open issues count, languages count, watchers count, missing values were replaced by zeros. This approach ensures that missing values do not skew the analysis of these numerical attributes.
 - In case of boolean columns, missing values were assigned the value "False".
- Handling the duplicate records
 - Duplicate records were identified and removed from the dataset. This process aimed to eliminate redundancy and ensure that each observation in the dataset was unique.
- Standardization of date formats
 - To ensure consistency, all date columns in the dataset were standardized.
 - This standardization was achieved by applying the Pandas `to_datetime` function, which converted date data into common and recognizable format across the dataset.
 - This facilitates accurate temporal analysis and visualization.
- Merging of multiple tables connected via repository/user IDs
 - Each user data was written into a CSV file based on the creation timestamp.

- This process was repeated 20 times over a span of 2 days to organize the user repositories data efficiently.
- Eventually, all the individual CSV files containing user data were merged into a single consolidated CSV file.

Furthermore, to enable geospatial analysis, we parsed free-text location fields to extract US state information. Non-US contributors were excluded from this specific angle of analysis. We also used the Microsoft PowerBI tool to create the heatmap of State Wise contributions.

Data Analysis

- We started our analysis by working with a meticulously cleaned dataset, ensuring data integrity and reliability.
- Beginning with descriptive statistics, we examined the data distributions and created informative visualizations. This step was facilitated by utilizing python libraries such as matplotlib, seaborn, and scipy. These initial analyses provided a comprehensive understanding of the dataset's characteristics and trends.
- Subsequently, we used various clustering algorithms including K-Means, Hierarchical, and DBSCAN, to categorize the US states into two distinct groups (Users and Organizations) where silhouette score was used to evaluate the clustering categories. This clustering process allowed us to identify the patterns and relationships within the dataset.
- To evaluate the significance of variation within the dataset, we applied ANOVA hypothesis testing, Chi-Square testing (⁷McHugh 2013). This statistical testing helped us to determine whether there were substantial differences in key metrics, such as programming languages and temporal patterns.

Results

Trends

Our analysis uncovered several interesting trends related to open source development:

- The C language dominates among individual contributors, used in over 20% of personal repositories. However, C++ is more popular within organizational accounts, utilized in 19% of their OSS projects. (Figure 1 and 2)
- California leads US states with 22% of all repositories, followed by New York, Texas, Washington, Illinois, and Massachusetts (Figure 3). Wyoming, North Dakota, and Delaware have the least contributions to the OSS projects. This is expected as the country's major Computer science innovation and business clusters are located in these states (⁴Moretti 2021)

- Clustering: Performed various clustering algorithms such as K-Means, Hierarchical, and DBSCAN on the data, accurately dividing users and organizations separately.
- Temporal patterns indicate contributor activity tailing off during holiday seasons and summer months. Additionally, organizational accounts exhibit more frequent code updates compared to individuals.
- Programming languages significantly impact per capita contributions - users working on Scala and Clojure projects contribute 2x more on average than those using Javascript or PHP.

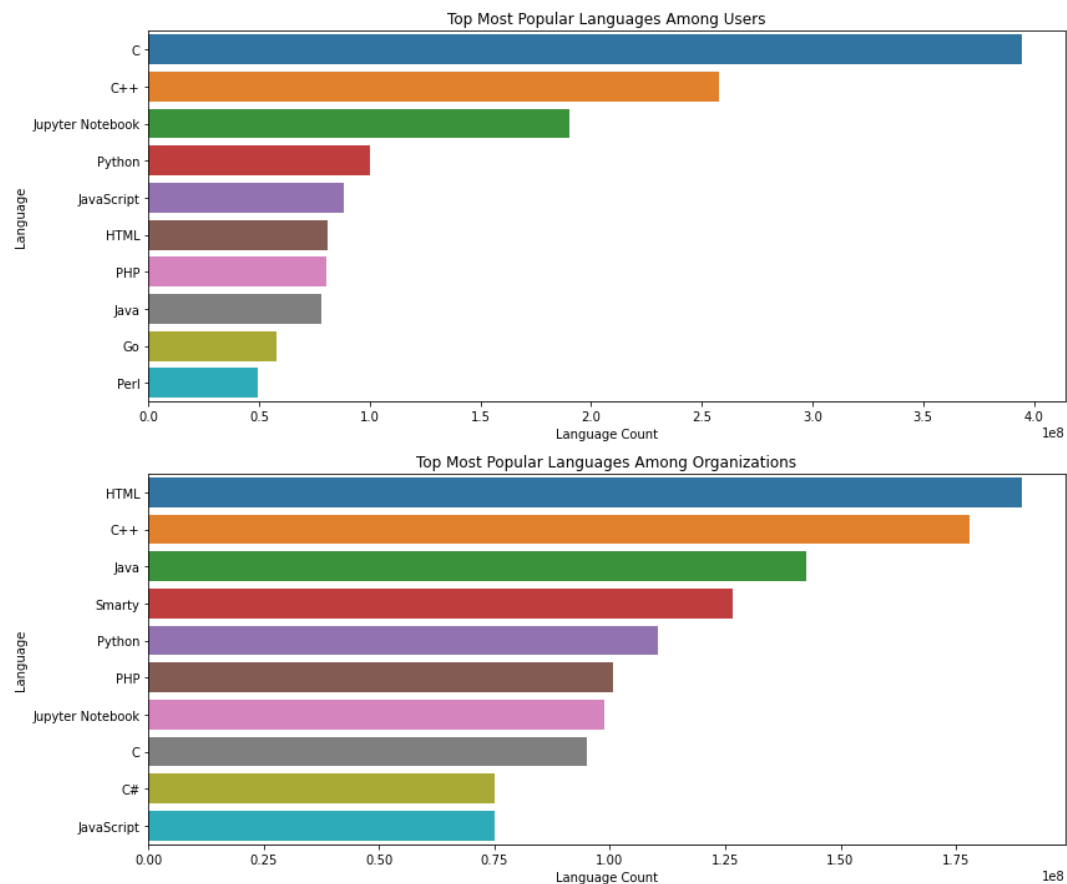


Figure 1: Language used in OSS projects (2014-2022)

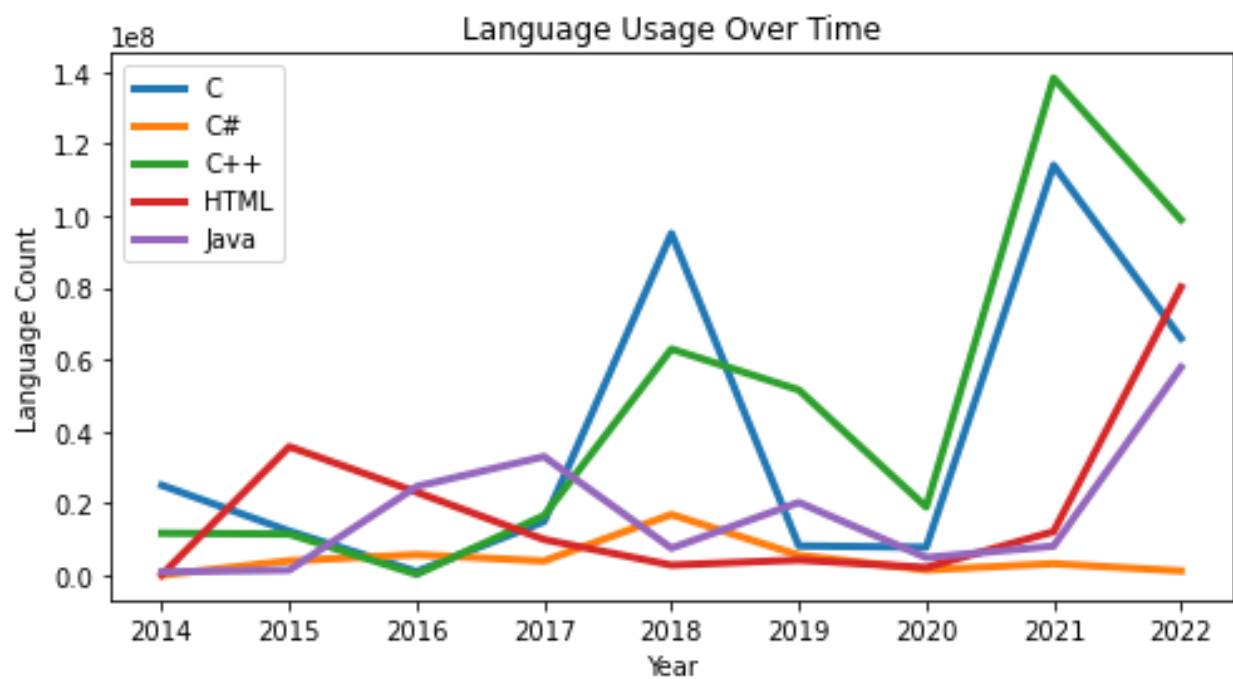


Figure 2: Usage of languages over time

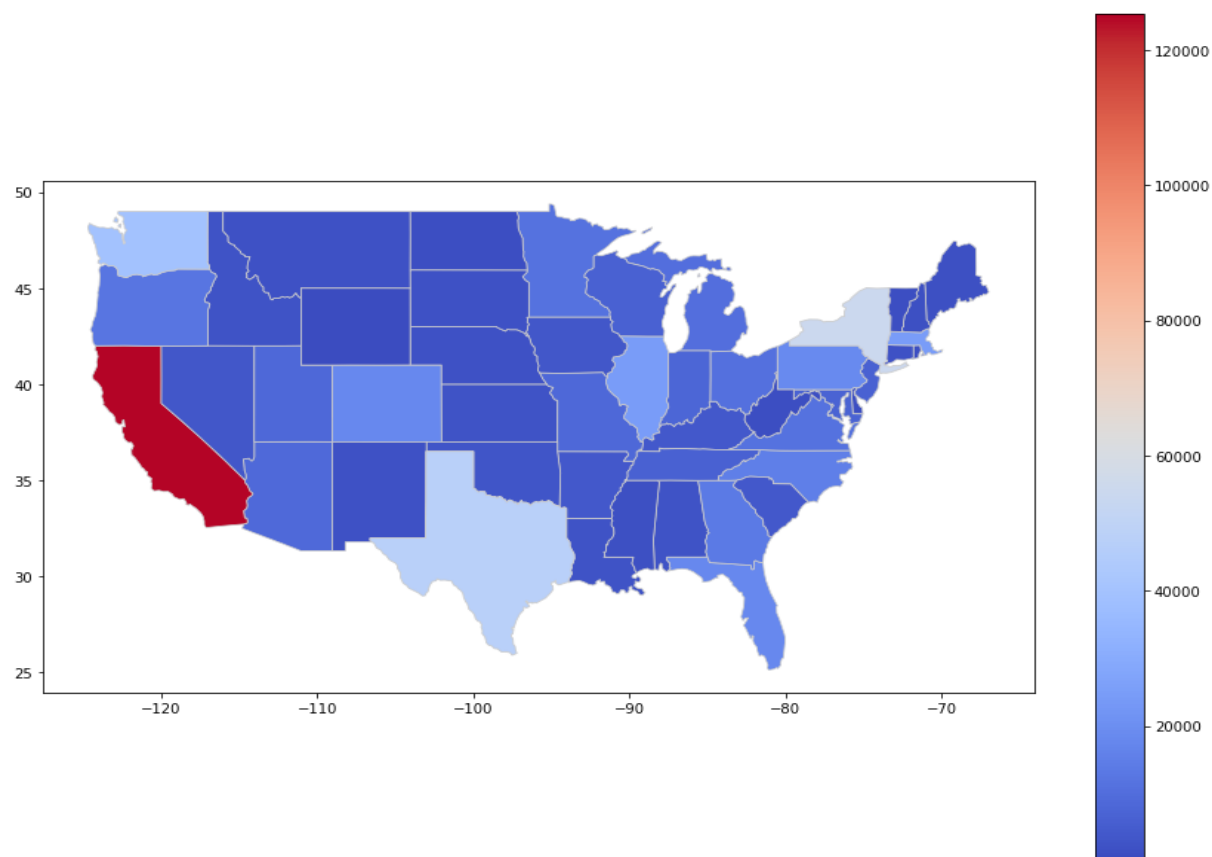


Figure 3: Heatmap of US State wise contributions

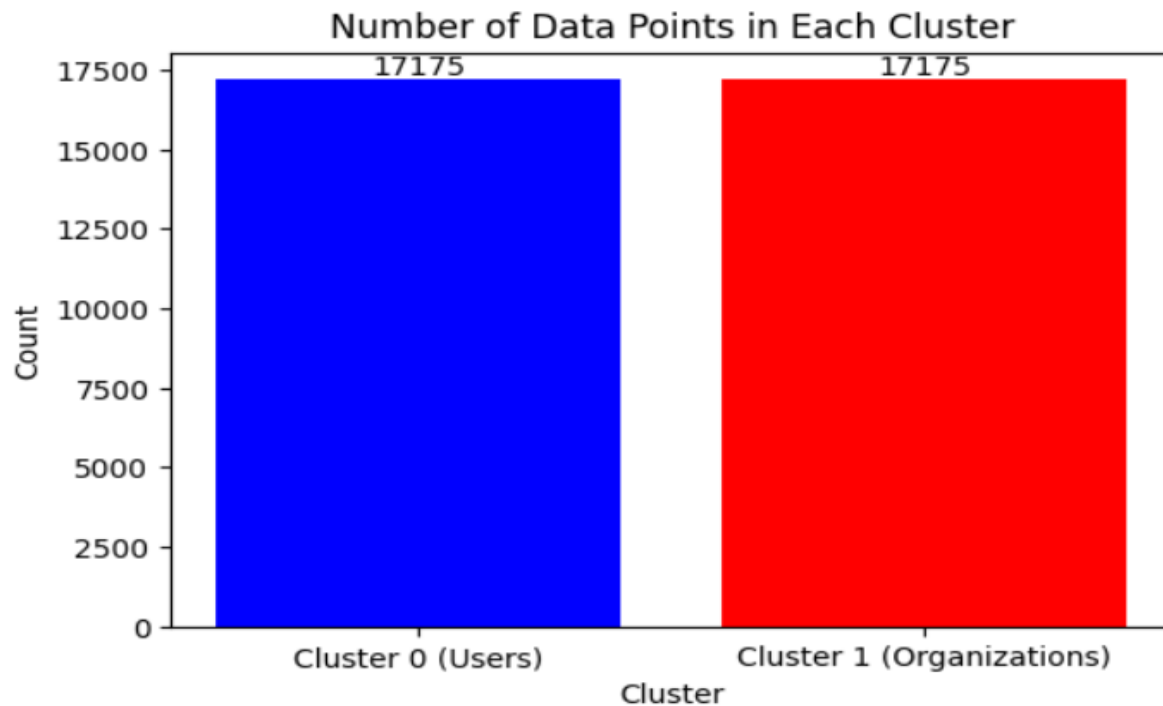


Figure 4: Clustering results for User and organizations

Hypothesis Testing

To supplement broader descriptive trend analysis, the team developed and evaluated the following focused hypotheses through statistical testing methods:

Hypothesis 1: Language Specificity in Contributions

H0 (Null): The average number of contributions per user is the same across different programming languages

H1 (Alternative): Certain programming languages are associated with significantly higher average contributions per user than others

This could indicate that some languages have more productivity than others.

Conclusion: The low p-values in the ANOVA test indicates that the average number of contributions per user is not the same across different programming languages among both individual and organization developers. (Table 1)

Table 1: ANOVA results for Hypothesis 1

<i>ANOVA</i>	<i>F-statistic</i>	<i>p-value</i>
Users	1.440	$3.208 * 10^{-07}$
Organizations	1.923	$8.997 * 10^{-11}$

Hypothesis 2: Temporal Activity Patterns in Repository Creation

H0 (Null): there are no significant differences in repository creation across different times of the year.

H1 (Alternative): There is variation in activity level during different months of a year.

Conclusion: The ANOVA test and the Heat map shows there is significant variation in the OSS contributions. This is perhaps because of holidays, academic cycles, industry trends, and events like hackathons.

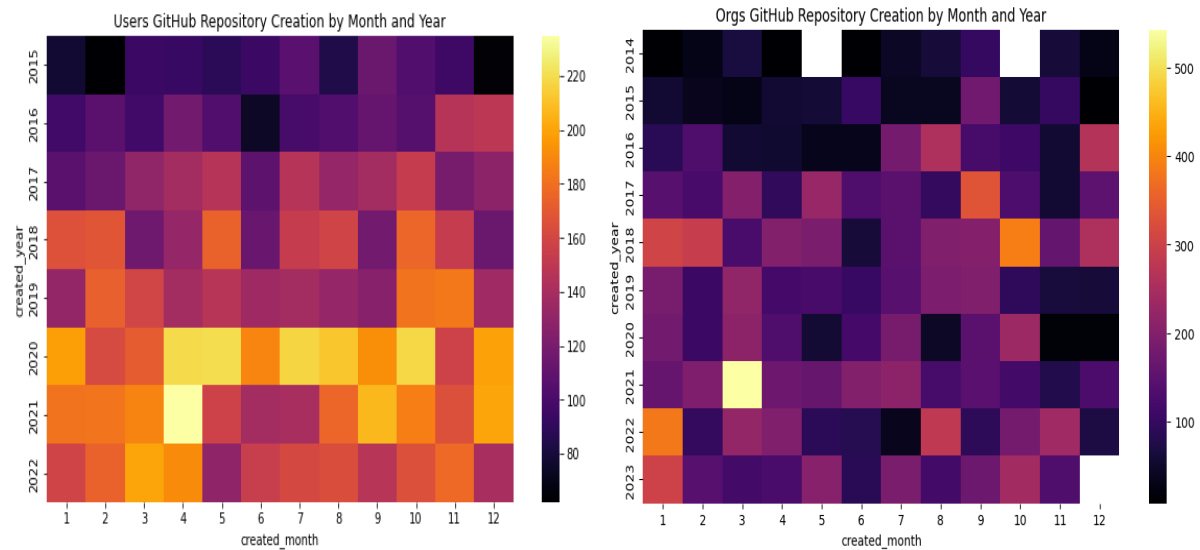


Figure 5: Heatmap of month-wise trend of contributions for users (left) and Organizations (Right)

Table 2: ANOVA results for Hypothesis 2

<i>ANOVA</i>	<i>F-statistic</i>	<i>p-value</i>
Users	2.36	0.0065
Organizations	24.56	$1.119 * 10^{-50}$

Hypothesis 3: Repository Activity and User Type Dynamics

H0 (Null): There is no significant difference in push or update frequencies between repositories owned by individual users and organizations.

H1 (Alternative): Organization-type users have repositories with more frequent activity (pushes and updates) than individual users, possibly due to more resources or collaborative efforts in organization

Observations:

Average days since last update for organizations: 926.237

Average days since last update for individual users: 1327.119

Average days since last push for organizations: 1234.44

Average days since last push for individual users: 1559.41

Conclusion: we have enough evidence to reject the Null Hypothesis as the p-value is lesser than the threshold value(0.05). It's evident from the t-test that the shorter time since the last push and update for organizations suggest that they are more active than users in terms of maintenance and updates.

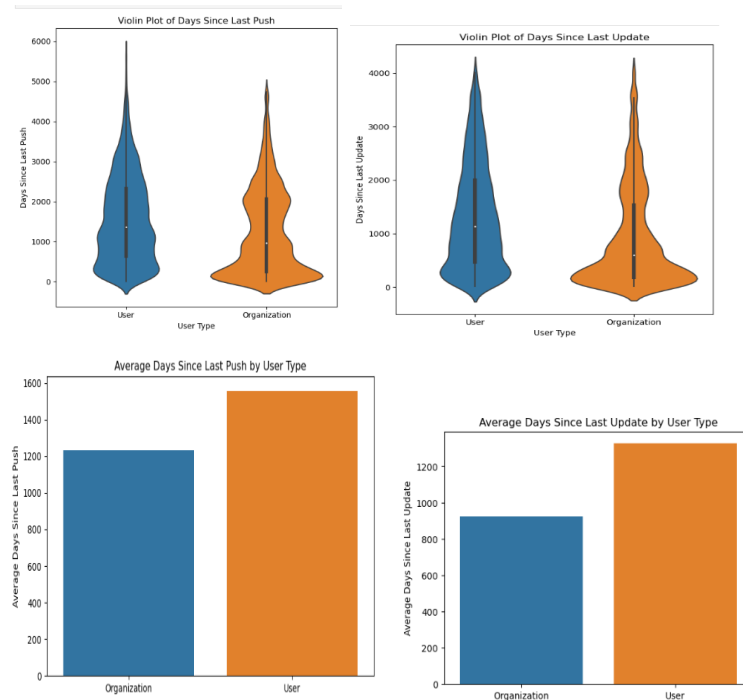


Figure 6: Various plots which compare push activity and update activity between users and organizations

Table 3: T- Test results for Hypothesis 3

<i>T-Test</i>	<i>t-statistic</i>	<i>p-value</i>
----------------------	---------------------------	-----------------------

Push Activity	-27.52	0.0
Update Activity	-38.50	0.0

Here, the negative sign indicates that organizations have a lower mean days since the last update/push compared to individual users.

Hypothesis 4: Are certain languages more popular in certain states?

H0 (Null): Certain programming languages are more popular in specific states due to regional factors.

H1 (Alternative): The popularity of programming languages is not affected by geographic regions

Conclusion: Chi-Square test p-value is approximately zero. It implies that certain programming languages are popular in specific states.

Table 4: Chi-Square Test results for Hypothesis-4

Chi-Square Test	p-value
For Users	0.6640
For Orgs	0.000

Table 5: Popularity of languages in different states

Programming Language	Popular States
C	Illinois, Colorado, Oregon, Indiana
C++	California, New York, Minneapolis
Jupyter Notebook	Illinois, Virginia, Michigan, Idaho
HTML	Illinois, Arizona, California
JAVA	Texas, California, Kansas, Michigan

Conclusion

Findings

Our analysis of open-source software (OSS) development projects contributions on GitHub uncovered many insights into trends of OSS and developer behaviors. Examining the usage rate of programming languages revealed the dominance of C and C++ among individual developers, likely due to its speed and proximity to hardware. However, organizations gravitate towards HTML and C++, likely due to web-focused projects and their scalability for complex projects. We also found that geography is also linked to technology hubs – California, New York, Texas, Washington, Illinois, and Massachusetts are hotspots attracting skilled labor through vibrant innovation ecosystems.

We also studied the temporal aspect of OSS contributions. Analyzing temporal patterns uncovered activity fluctuations aligned to real-world events like holiday lulls. We also found that organizational contributors are faster at closing open issues than individual developers. It could be balancing personal needs – having requisite staffing resources clearly matters.

Limitations and Future Studies

While we took a data-driven approach to study real-world OSS contribution dynamics, there were a few limitations. As the data was extremely large, where contribution level observations were in billions and project level in millions, we focused on a few potential trends. We created an aggregated subsample of the big data to create visualizations of this study. While we did an observational study, we cannot make causal conclusions without further analysis. We believe more suitable econometric techniques should be used to isolate causal effects. Additionally, we acknowledge that the public GitHub dataset, while robust, does not capture the entire open-source landscape. However, as it is the biggest platform for OSS repositories, it can be used for further studies.

References

1. Von Krogh, G., Haefliger, S., Spaeth, S., & Wallin, M. W. (2012). Carrots and rainbows: Motivation and social practice in open source software development. *MIS quarterly*, 649-676.
2. Davenport, T., Bean Raddy. (2022), Companies Are Making Serious Money With AI, MIT Sloan Review, 2022.
<https://sloanreview.mit.edu/article/companies-are-making-serious-money-with-ai/>
3. Han, J., Kamber, M., & Mining, D. (2006). Concepts and techniques. Morgan kaufmann, 340, 94104-3205.
4. Moretti, E. (2021). The effect of high-tech clusters on the productivity of top inventors. *American Economic Review*, 111(10), 3328-3375.
5. Yu, C.H., (2010), Exploratory data analysis in the context of data mining and resampling. *Int. J. Psychol. Res.* 2010,3, 9–22. [Exploratory data analysis in the context of data mining and resampling](#)
6. Chandrakantha, L, (2014). Learning ANOVA concepts using simulation, Proceedings of the 2014 Zone 1 Conference of the American Society for Engineering Education, Bridgeport, CT, USA, 2014, pp. 1-5, doi: 10.1109/ASEEZone1.2014.6820644. [Learning ANOVA concepts using simulation](#)
7. McHugh ML. (2013). The chi-square test of independence. *Biochem Med (Zagreb)*. 2013;23(2):143-9. doi: 10.11613/bm.2013.018. PMID: 23894860; PMCID: PMC3900058. [The Chi-square test of independence](#)

Presentation YouTube video:

[CSCI 5502 Fall 2023 - Group3 PPT Shashi & Venkata \(youtube.com\)](#)